

Fuentes de Datos · Práctica de Docker

Contesta primero sin computadora, como en el examen. Después puedes correrlo todo en tu máquina y comparar: así se comprobó la clave.

Ejercicio 1 · Lee el Dockerfile

encuestas/ es un repo de Python. El programa limpiar.py usa reglas.py, lee crudos/respuestas.csv y escribe salida/limpio.csv. Las dos rutas son relativas a la carpeta donde corre.

```
encuestas/      ← estás aquí
├── .dockerignore
├── Dockerfile
├── README.md
├── requirements.txt
├── limpiar.py
├── reglas.py
├── notas/
│   └── ideas.md
├── crudos/
│   └── respuestas.csv
└── salida/
```

El .dockerignore:

```
crudos/
notas/
salida/
```

El Dockerfile:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "limpiar.py"]
```

La imagen ya se construyó una vez con `docker build -t encuestas:1 ..`

a) El `COPY . .` de la línea 5 ya copia `requirements.txt`. ¿Para qué se copia antes, por separado, en la línea 3?

b) ¿Qué archivos y carpetas quedan dentro de `/app` en la imagen?

c) Escribe el comando que corre `encuestas:1` en un contenedor que se borre al terminar, con tu carpeta `crudos/` montada en `/app/crudos` y tu carpeta `salida/` montada en `/app/salida`.

d) Alguien corre `docker run --rm encuestas:1`, sin montar nada. ¿Qué pasa y por qué?

e) Después del primer build cambias **un solo** archivo y vuelves a construir. Para cada caso, ¿qué pasos se rehacen y cuáles salen de la caché?

1. Agregas una línea a `notas/ideas.md`.
2. Cambias `reglas.py`.
3. Agregas un paquete a `requirements.txt`.
4. Corriges un typo en `README.md`.

f) Alguien borra la línea `salida/` del `.dockerignore`. Corre el programa con la salida montada, como en c), y vuelve a construir. ¿Qué dos cosas cambian respecto de antes?

g) `docker run --rm encuestas:1 python reglas.py`, ¿corre `limpiar.py`? ¿Por qué?

Ejercicio 2 · ¿Qué cambió?

En `lab/` hay sólo este Dockerfile y `saludo.txt`, cuyo texto es la palabra **hola**:

```
FROM alpine:3.20
WORKDIR /s
COPY saludo.txt .
CMD ["cat", "saludo.txt"]
```

No existen imágenes, contenedores, volúmenes ni caché previos. Los comandos se corren **en orden**, todos desde `lab/`; ningún `sleep 600` alcanza a terminar. Cada fila parte de lo que dejó la anterior. Contesta con una palabra o frase corta, o **error** si crees que un comando falla.

Fila 1.

```
docker build -t saludo:1 .
docker run -d --name pa saludo:1 sleep 600
docker exec pa sh -c 'echo adios > saludo.txt'
docker stop pa
docker start pa
docker exec pa cat saludo.txt
```

¿Qué imprime el último `exec`?

Respuesta: _____

Fila 2.

```
docker run --rm saludo:1
```

¿Qué imprime?

Respuesta: _____

Fila 3.

```
docker run -d --name pb -v caja:/s saludo:1 sleep 600
```

```
docker exec pb sh -c 'echo luna > saludo.txt'
```

```
docker run --rm -v caja:/s saludo:1
```

¿Qué imprime el último run?

Respuesta: _____

Fila 4.

```
echo sol > saludo.txt
```

```
docker build -t saludo:1 .
```

```
docker run --rm -v caja:/s saludo:1
```

¿Qué imprime el run?

Respuesta: _____

Fila 5.

```
docker run --rm saludo:1
```

¿Qué imprime?

Respuesta: _____

Fila 6.

```
docker exec pb cat saludo.txt
```

¿Qué imprime?

Respuesta: _____

Fila 7.

```
docker rm -f pb
```

```
docker volume rm caja
```

```
docker run --rm -v caja:/s saludo:1
```

¿Qué imprime el run?

Respuesta: _____

Fila 8.

```
docker exec pa cat saludo.txt
```

¿Qué imprime?

Respuesta: _____

Fila 9.

```
echo nube > saludo.txt
```

```
docker run --rm -v "$(pwd)":/s saludo:1
```

```
docker run --rm saludo:1
```

¿Qué imprime el primer run? ¿Y el segundo?

Respuesta: _____

Fila 10.

```
docker build -t saludo:1 .
```

El paso COPY, ¿sale de la caché o se rehace? ¿Por qué?

Respuesta: _____

Fila 11.

```
docker run --rm saludo:1 sh -c 'echo x > saludo.txt; cat saludo.txt'
docker run --rm saludo:1
```

¿Qué imprime el primer run? ¿Y el segundo?

Respuesta: _____

Fila 12.

```
docker rm -f pa
```

Los textos **adios** y **luna**, ¿dónde existen ahora? (tu carpeta / la imagen / un volumen / en ningún lado)

Respuesta: _____

Las filas 9 y 11 tienen dos casillas: 14 en total.

Ejercicio 3 · Lee el error

Cada caso trae el comando y lo que contestó Docker. Escribe **qué pasó** en una línea y **qué harías**.

1. Justo después de la fila 1 del ejercicio 2, alguien vuelve a correr la misma línea:

```
$ docker run -d --name pa saludo:1 sleep 600
docker: Error response from daemon: Conflict. The container name "/pa" is already in use by
↳ container "94ec18da7a0a...". You have to remove (or rename) that container to be able to
↳ reuse that name.
```

2.

```
$ docker build -t saludo:1
ERROR: docker: 'docker buildx build' requires 1 argument
```

Usage: docker buildx build [OPTIONS] PATH | URL | -

3. Después de `docker stop pa`:

```
$ docker exec pa cat saludo.txt
Error response from daemon: container 94ec18da7a0a... is not running
```

4.

```
$ docker run --rm -it saludo:1 bash
docker: Error response from daemon: failed to create task for container: ... exec: "bash":
↳ executable file not found in $PATH
```

5. Un Dockerfile en proyecto/app/ con la línea COPY ../datos /datos, construido desde proyecto/app/:

```
$ docker build -t app:1 .
ERROR: failed to build: failed to solve: failed to compute cache key: ... "/datos": not found
```